

Proyecto Final — Validación y Verificación de Software

Hospital Management System

Escuela Politécnica Nacional
Facultad de Ingeniería de Sistemas
Período Académico: 2026A

Contenido

1. [Introducción](#)
2. [Descripción del Sistema](#)
3. [Arquitectura del Proyecto](#)
4. [Historias de Usuario](#)
5. [Modelo de Datos](#)
6. [Guía de Instalación](#)
7. [Detalle de Entregables](#)
8. [Rúbrica de Evaluación](#)
9. [Recomendaciones](#)

1. Introducción

El presente documento describe el proyecto final de la materia **Validación y Verificación de Software**. El objetivo es que los estudiantes apliquen de manera práctica los conceptos aprendidos durante el semestre sobre pruebas de software, análisis estático y seguridad en un sistema fullstack real.

1.1 Objetivos de Aprendizaje

Al completar este proyecto, el estudiante será capaz de:

- Diseñar e implementar **pruebas unitarias** para capas de servicio (backend) y módulos de utilidad (frontend)
- Diseñar e implementar **pruebas de integración** para APIs REST y flujos de usuario
- Configurar y ejecutar herramientas de **análisis estático de código**
- Identificar vulnerabilidades de seguridad según el estándar **OWASP Top 10**
- Trabajar colaborativamente en un proyecto de software aplicando buenas prácticas de **DevOps y CI/CD**
- Documentar hallazgos y proponer soluciones fundamentadas

1.2 Modalidad de Trabajo

- **Grupos de 3 a 4 estudiantes**
- El código base es proporcionado por el profesor
- Las pruebas, análisis y documentación deben ser desarrollados por cada grupo
- La entrega se realiza mediante un repositorio Git (GitHub/GitLab)

2. Descripción del Sistema

2.1 Contexto

El **Hospital Management System** es una aplicación web para la gestión administrativa de un hospital. Permite registrar pacientes, doctores, programar citas médicas y mantener historiales clínicos.

2.2 Funcionalidades

Módulo	Funcionalidades
Pacientes	CRUD, búsqueda por nombre, estadísticas de edad
Doctores	CRUD, búsqueda por especialidad y nombre
Citas Médicas	CRUD, filtrado por paciente/doctor/estado/rango de fechas
Historias Clínicas	Creación, consulta por paciente y doctor
Dashboard	Resumen de pacientes, doctores, citas del día

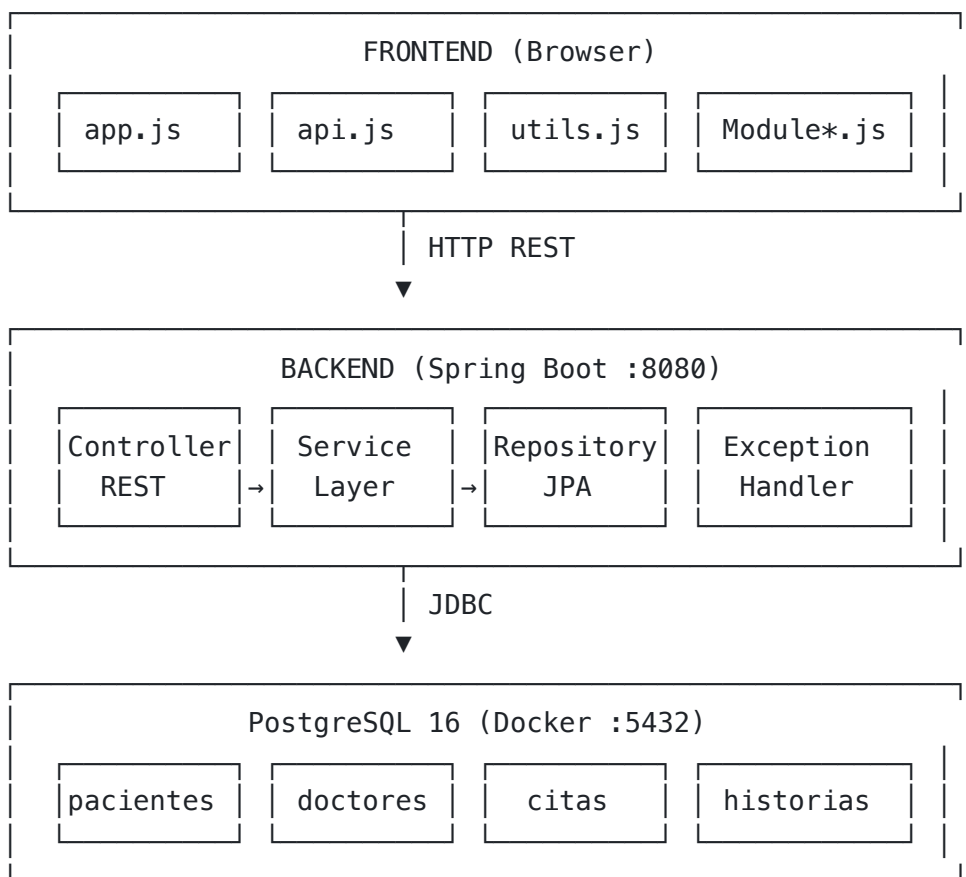
2.3 Tecnologías Utilizadas

El sistema utiliza un stack moderno de desarrollo web:

- **Backend:** Spring Boot 3.3 con Spring Data JPA, Jakarta Validation
- **Frontend:** JavaScript vanilla (modular), HTML5, CSS3
- **Base de Datos:** PostgreSQL 16 (contenedor Docker)
- **Build:** Maven (backend), sin bundler (frontend vanilla)

3. Arquitectura del Proyecto

3.1 Diagrama de Componentes



3.2 Patrón de Diseño

El backend sigue el patrón **MVC (Model-View-Controller)** con una capa de servicio:

- **Model:** Entidades JPA que mapean a las tablas de la base de datos
- **DTO:** Objetos de transferencia de datos para la comunicación REST
- **Repository:** Interfaces de acceso a datos con Spring Data JPA
- **Service:** Lógica de negocio y orquestación
- **Controller:** Endpoints REST y manejo de peticiones HTTP

El frontend sigue un patrón **modular** donde cada archivo JavaScript es responsable de una entidad o funcionalidad específica.

4. Historias de Usuario

HU-01: Registrar Paciente

Como administrador del hospital

Quiero registrar un nuevo paciente con sus datos personales

Para mantener un registro actualizado de pacientes

- **Criterios de aceptación:**
 - El formulario debe validar campos obligatorios: nombre, apellido
 - El email debe tener formato válido
 - El teléfono debe tener 10 dígitos
 - La fecha de nacimiento debe ser en el pasado
 - El paciente creado debe aparecer en la tabla de pacientes

HU-02: Gestionar Doctores

Como administrador del hospital

Quiero registrar y mantener la información de los doctores

Para asignarlos a citas y consultas

- **Criterios de aceptación:**
 - Se debe registrar nombre, apellido, especialidad, email, teléfono y consultorio
 - La especialidad es un campo requerido (aunque el sistema actual no lo valida)

HU-03: Agendar Citas Médicas

Como administrador del hospital

Quiero programar citas entre pacientes y doctores

Para organizar la atención médica

- **Criterios de aceptación:**

- La cita debe asociar un paciente y un doctor existentes
- La fecha y hora deben ser futuras
- No se debe permitir doble booking (mismo doctor, misma hora)

HU-04: Registrar Historia Clínica

Como doctor

Quiero registrar diagnósticos y tratamientos en la historia clínica del paciente

Para mantener un historial médico completo

- **Criterios de aceptación:**

- El diagnóstico es obligatorio
- La historia debe asociarse a un paciente existente
- Opcionalmente puede asociarse a un doctor

HU-05: Dashboard de Resumen

Como administrador

Quiero ver un resumen de la actividad del hospital

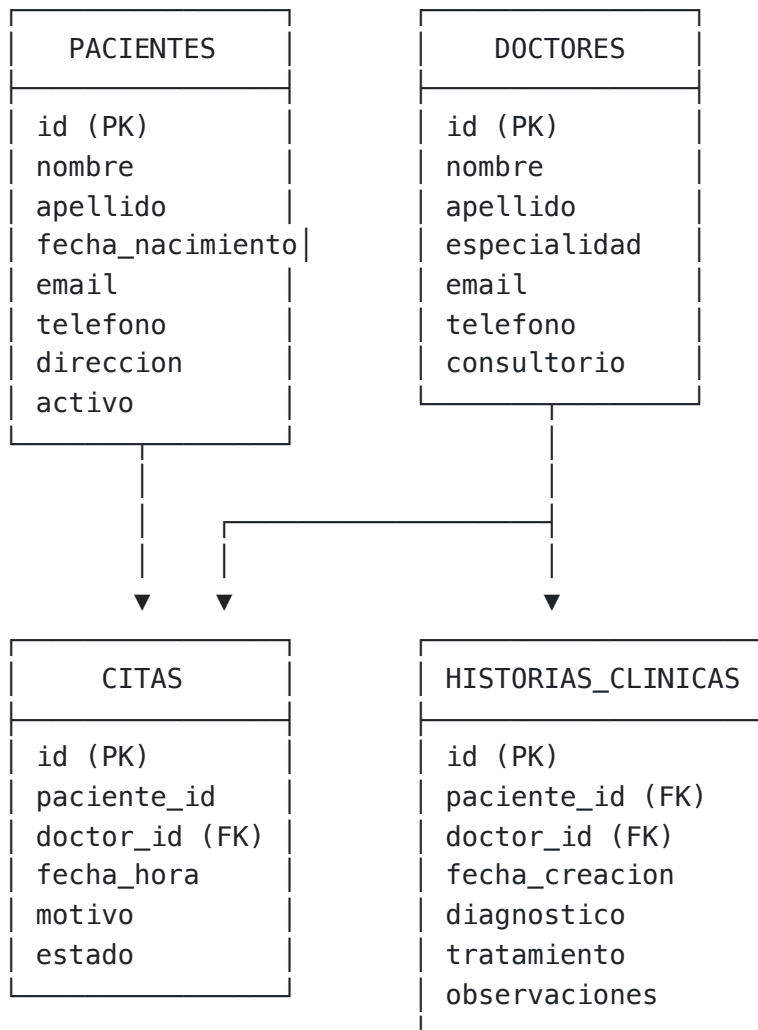
Para monitorear el estado general

- **Criterios de aceptación:**

- Mostrar total de pacientes registrados
- Mostrar total de doctores
- Mostrar citas programadas para el día actual
- Mostrar edad promedio de pacientes

5. Modelo de Datos

5.1 Diagrama Entidad-Relación



5.2 Notas sobre el Esquema

El esquema de base de datos contiene **imperfecciones intencionales** que los estudiantes deben identificar:

- La tabla `citas` **no tiene FOREIGN KEY** hacia `pacientes`
- La tabla `doctores` permite valores **NULL en especialidad** (no tiene restricción NOT NULL)
- Estas decisiones de diseño representan **deuda técnica** que debe ser detectada

6. Guía de Instalación

6.1 Requisitos del Sistema

Software	Versión Mínima	Notas
Java JDK	17	OpenJDK o Oracle JDK
Maven	3.9	Incluido como Maven Wrapper
Docker	24+	Con Docker Compose
Node.js	20+	Para ESLint y Jest
Navegador Web	Chrome/Firefox	Última versión

6.2 Pasos de Instalación

Paso 1: Clonar el repositorio

```
git clone <url-del-repositorio>  
cd hospital-management
```

Paso 2: Iniciar la base de datos

```
docker-compose up -d  
  
# Verificar que PostgreSQL esta corriendo  
docker ps | grep hospital-db  
  
# Los scripts schema.sql y data.sql se ejecutan automaticamente
```

Paso 3: Compilar y ejecutar el backend

```
cd backend  
  
# Linux/Mac  
./mvnw spring-boot:run  
  
# Windows  
mvnw.cmd spring-boot:run
```

El backend inicia en `http://localhost:8080` . Para verificar:

```
curl http://localhost:8080/api/pacientes
```

Paso 4: Abrir el frontend

Opción A — Archivo directo:

```
open frontend/index.html # Mac
start frontend/index.html # Windows
```

Opción B — Servidor HTTP:

```
cd frontend
python3 -m http.server 3000
# Abrir http://localhost:3000
```

6.3 Verificación de la Instalación

1. Abrir el frontend en el navegador
2. El dashboard debe mostrar contadores con datos
3. Navegar a Pacientes → debe mostrar 5 pacientes precargados
4. Navegar a Doctores → debe mostrar 4 doctores precargados
5. Navegar a Citas → debe mostrar 5 citas precargadas
6. Navegar a Historias Clínicas → debe mostrar 3 registros precargados

7. Detalle de Entregables

7.1 Pruebas Unitarias — Backend (30%)

Objetivo: Verificar el correcto funcionamiento de la capa de servicio de forma aislada.

Herramientas: JUnit 5, Mockito

Entregable: Directorio `backend/src/test/java/com/hospital/service/` con las siguientes clases de prueba:

Clase de Prueba	Servicio bajo prueba	Puntos
PacienteServiceTest	PacienteService	8
DoctorServiceTest	DoctorService	8
CitaServiceTest	CitaService	7
HistoriaClinicaServiceTest	HistoriaClinicaService	7

Requisitos mínimos por clase de prueba:

- Al menos 3 pruebas de **caso feliz** (happy path)
- Al menos 2 pruebas de **casos límite** (boundary testing)
- Al menos 2 pruebas de **manejo de errores**
- Uso correcto de **mocks** para dependencias
- Cobertura mínima del 80% en capa de servicio

7.2 Pruebas Unitarias — Frontend (15%)

Objetivo: Verificar funciones JavaScript de forma aislada.

Herramientas: Jest

Entregable: Archivos de prueba en `frontend/js/__tests__/` :

Archivo de Prueba	Módulo bajo prueba	Puntos
<code>utils.test.js</code>	<code>utils.js</code>	10
<code>api.test.js</code>	<code>api.js</code> (mock)	5

Requisitos mínimos:

- Probar todas las funciones exportadas de `utils.js`
- Probar escenarios con entradas válidas e inválidas
- Probar casos límite (null, undefined, strings vacíos)

- Mock de `fetch` para probar `api.js`

7.3 Pruebas de Integración — Backend (20%)

Objetivo: Verificar la integración entre capas (Controller → Service → Repository → DB).

Herramientas: Spring Boot Test, MockMvc, H2 Database

Entregable: Directorio `backend/src/test/java/com/hospital/controller/` con:

Clase de Prueba	Puntos
<code>PacienteControllerIntegrationTest</code>	6
<code>DoctorControllerIntegrationTest</code>	5
<code>CitaControllerIntegrationTest</code>	5
<code>HistoriaClinicaControllerIntegrationTest</code>	4

Requisitos:

- Uso de `@SpringBootTest` con H2 en memoria
- Pruebas de endpoints GET, POST, PUT, DELETE
- Verificación de códigos de estado HTTP
- Verificación de cuerpos de respuesta JSON

7.4 Pruebas de Integración — Frontend (15%)

Objetivo: Verificar flujos completos de usuario en el navegador.

Herramientas: Playwright o Selenium WebDriver

Entregable: Scripts de prueba en `frontend/e2e/` que cubran:

Escenario	Puntos
Flujo CRUD de pacientes	4
Flujo CRUD de doctores	3
Flujo de creación y consulta de citas	4
Flujo de creación y consulta de historias	4

7.5 Análisis Estático de Código (10%)

Objetivo: Identificar problemas de calidad de código sin ejecutarlo.

Herramientas sugeridas:

Backend:

- SonarQube Community Edition (recomendado)
- SpotBugs + FindSecBugs
- Checkstyle (Google Java Style)

Frontend:

- ESLint con reglas recomendadas

Entregable: Informe PDF que incluya:

- Herramientas utilizadas y su configuración
- Lista de hallazgos clasificados por severidad (Blocker, Critical, Major, Minor)
- Capturas de pantalla de los resultados
- Análisis de al menos 5 hallazgos relevantes

7.6 Análisis de Seguridad OWASP (10%)

Objetivo: Identificar vulnerabilidades de seguridad según el estándar OWASP Top 10 (2021).

Entregable: Informe PDF que incluya:

1. **Mapeo de vulnerabilidades encontradas** al OWASP Top 10
2. **Descripción detallada** de cada vulnerabilidad:
 - Ubicación (archivo, línea)
 - Categoría OWASP
 - Riesgo (Alto/Medio/Bajo)
 - Impacto potencial
 - Evidencia (screenshot o código)
3. **Recomendaciones de mitigación** para cada hallazgo
4. **Verificación** — ¿cómo comprobar que la vulnerabilidad existe?

8. Rúbrica de Evaluación

8.1 Pruebas Unitarias Backend (30 puntos)

Aspecto	Excelente (100%)	Bueno (75%)	Aceptable (50%)	Insuficiente (25%)
Cobertura de casos felices	3+ por método	2 por método	1 por método	Sin pruebas
Casos límite	Identifica 2+	Identifica 1	Intenta	No hay
Manejo de errores	2+ escenarios	1 escenario	Intenta	No hay
Uso de mocks	Correcto y aislado	Correcto	Parcial	Sin mocks
Cobertura de código	> 85%	75-84%	60-74%	< 60%

8.2 Pruebas Unitarias Frontend (15 puntos)

Aspecto	Excelente	Bueno	Aceptable	Insuficiente
Cobertura utils.js	Todas las funciones	80%	50%	< 50%
Casos con datos inválidos	2+ por función	1 por función	Intenta	No hay
Mock de fetch	Completo y realista	Correcto	Parcial	No hay

8.3 Pruebas de Integración Backend (20 puntos)

Aspecto	Excelente	Bueno	Aceptable	Insuficiente
CRUD completo por entidad	4 ops	3 ops	2 ops	< 2 ops

Verificación HTTP status	Siempre	Casi siempre	A veces	Nunca
Verificación body JSON	Completa	Parcial	Superficial	No verifica
Aislamiento (H2)	Correcto	Correcto	Parcial	Usa PostgreSQL

8.4 Pruebas de Integración Frontend (15 puntos)

Aspecto	Excelente	Bueno	Aceptable	Insuficiente
Flujos completos	4 flujos	3	2	< 2
Manejo de esperas	Correcto	Bueno	Regular	Sin esperas
Screenshots/evidencia	Sí	Sí	Parcial	No

8.5 Análisis Estático (10 puntos)

Aspecto	Excelente	Bueno	Aceptable	Insuficiente
Herramientas configuradas	3+ herramientas	2	1	Ninguna
Hallazgos reportados	> 15	10-14	5-9	< 5
Clasificación severidad	Correcta	Casi correcta	Parcial	No clasifica

8.6 Análisis OWASP (10 puntos)

Aspecto	Excelente	Bueno	Aceptable	Insuficiente
Vulnerabilidades encontradas	8+	5-7	3-4	< 3
Mapeo a OWASP Top 10	Preciso	Casi preciso	Parcial	Incorrecto

Recomendaciones	Concretas y viables	Viables	Genéricas	No hay
-----------------	---------------------	---------	-----------	--------

9. Recomendaciones

9.1 Para el Desarrollo

1. **Comiencen por entender el código base.** Lean el código, ejecuten la aplicación y naveguen por todas las funcionalidades antes de empezar a escribir pruebas.
2. **Sigan el orden sugerido:**
 - Primero: Pruebas unitarias backend (dan más puntos y son la base)
 - Segundo: Pruebas de integración backend
 - Tercero: Pruebas unitarias frontend
 - Cuarto: Pruebas de integración frontend
 - Paralelamente: Análisis estático y de seguridad
3. **División del trabajo recomendada:**
 - Estudiante A: Pruebas unitarias backend (PacienteService, DoctorService)
 - Estudiante B: Pruebas unitarias backend (CitaService, HistoriaClinicaService) + integración
 - Estudiante C: Pruebas frontend + ESLint
 - Estudiante D: Análisis estático backend + OWASP
4. **Usen control de versiones.** Hagan commits frecuentes y significativos. Usen ramas por funcionalidad y pull requests para revisar el código entre ustedes.

9.2 Para la Entrega

El repositorio debe contener:

```

grupo-XX/
├── backend/
│   └── src/test/           # Pruebas implementadas
├── frontend/
│   └── js/__tests__/_/     # Pruebas unitarias frontend

```

```
|   └─ e2e/                # Pruebas end-to-end
|   └─ informes/
|       └─ analisis-estatico.pdf
|       └─ analisis-owasp.pdf
|       └─ cobertura/      # Reportes de cobertura
|   └─ .github/workflows/  # CI/CD (puntos extra)
|   └─ README.md           # Documentación del grupo
```

9.3 Fechas Importantes

Hito	Fecha
Publicación del proyecto	Semana 12
Revisión de avance (50%)	Semana 14
Entrega final	Semana 16
Sustentaciones	Semana 17

Apéndice A: Comandos Útiles

Backend

```
# Ejecutar todas las pruebas
mvn test

# Ejecutar una clase de prueba especifica
mvn test -Dtest=PacienteServiceTest

# Generar reporte de cobertura con JaCoCo
mvn test jacoco:report

# Verificar estilo de código con Checkstyle
mvn checkstyle:check
```

Frontend

```
# Instalar Jest
npm init -y
npm install --save-dev jest

# Ejecutar pruebas
npx jest

# Ejecutar ESLint
npx eslint js/**/*.js

# Instalar Playwright
npm install --save-dev @playwright/test
npx playwright install
```

Docker

```
# Ver logs de PostgreSQL
docker logs hospital-db

# Conectarse a la base de datos
docker exec -it hospital-db psql -U admin -d hospital

# Reiniciar la base de datos (borra todos los datos)
docker-compose down -v
docker-compose up -d
```

Apéndice B: Preguntas Frecuentes

P: ¿Podemos modificar el código base?

R: Pueden agregar pruebas y configuraciones, pero no deben modificar el código fuente (a menos que sea para el punto extra de correcciones).

P: ¿Qué pasa si encontramos un bug que no está en la lista?

R: Documentenlo. Si es un bug real, recibirán puntos extra.

P: ¿Podemos usar IntelliJ IDEA Community Edition?

R: Sí, cualquier IDE que soporte proyectos Maven funciona.

P: ¿Las pruebas de integración deben usar PostgreSQL real?

R: No. Deben usar H2 en memoria para que las pruebas sean reproducibles.

Escuela Politécnica Nacional — Validación y Verificación de Software — 2026A

Profesor: Christian Suárez